

PALMPAY CLONE

Complete Code Guide

Every screen, class and flow in this repository, explained in plain terms - with a where-to-edit cheat sheet. Java + XML, MVC-style controllers, immutable models, repository data sources.

Contents

0. Big picture	2
1. Home screen	2
Where to edit	2
2. Profile (customisation form)	2
Where to edit	3
3. Transfer to Bank	3
3.1 Account input	3
3.2 Matching banks request cycle - showMatchingBanks()	3
3.3 Selecting a bank	3
3.4 Support classes	3
4. Bank picker	3
5. Amount screen	4
Where to edit	4
6. Add Money (card funding)	4
6.1 The silent destination gate	4
6.2 Route 1 - Paystack (default)	4
6.3 Route 2 - Stripe	4
6.4 Route 3 - Flutterwave	4
6.5 Browser handling on low-RAM devices	4
Where to edit	5
7. Persistence policy	5
8. Themes	5
9. Tests & CI	5
10. Cheat sheet - where do I change...?	5

0. Big picture

The app is Java + XML (no Kotlin, no Compose). Each screen has an Activity (the shell) plus a Controller (all behaviour). Data lives in repositories and stores, and screens render immutable model objects built by catalogues - so replacing demo data with a real API later only touches the data classes.

.MainActivity	Home screen shell: view binding, system bars.
.ui.HomeScreenController	All home behaviour and carousels.
.data.WalletStore	SharedPreferences: balance, name, key.
.data.HomeCatalog	Static home content as models.
.model.*	QuickAction, ServiceAction, PromotionCard.
.profile.*	ProfileActivity + controller (customisation).
.transfer.ui.*	Transfer, Amount, BankPicker screens.
.transfer.data.*	Paystack, Stripe, Flutterwave, NUBAN, logos, names.
.transfer.model.*	BankInstitution, TransferRecipient, ...

Layouts live in res/layout (one XML per screen and per row); light/dark values in res/values and res/values-night; XML shapes in res/drawable and PNG art in res/drawable-nodpi.

1. Home screen

MainActivity.onCreate builds its ViewBinding and calls HomeScreenController.bind(), which renders each region in order:

- renderQuickActions() - To Bank / To PalmPay / Savings / Cards; To Bank opens TransferActivity.
- renderServices() - the 8-tile grid; the Data tile carries the orange Promo badge.
- renderPromotions() - Team Up & Save and CashBox cards.
- bindBalanceCard() - WalletStore balance with hide/show eye; Add Money / History are toasts.
- bindHeader() - greeting 'Hi, <name>' from WalletStore; avatar opens Profile.
- bindPromoCarousel() / bindBannerCarousel() - ViewFlippers auto-flipping every 4000 ms with fades; dots synced by a Handler loop.
- bindNavigation() - custom bottom bar (icon masks tinted purple/ink, red dot on Loan, NEW pill on Wealth).

Where to edit

- Tile texts: res/values/strings.xml; which tiles exist: data/HomeCatalog.java.
- Tap behaviour: ui/HomeScreenController.java; carousel speed: the setFlipInterval(4000) calls.
- Colours per theme: res/values/colors.xml + res/values-night/colors.xml.

2. Profile (customisation form)

One Save button persists every field into the 'palmpay_clone_wallet' SharedPreferences, and Home re-reads the balance in onResume().

Balance	saveBalance() - the number the home card shows.
Display Name	saveDisplayName() - drives the home greeting.
Paystack key	savePaystackApiKey() - name lookup + Add Money route 1.
Stripe key	saveStripeApiKey() - Add Money route 2.
Flutterwave key	saveFlutterwaveApiKey() - Add Money route 3.
Flutterwave enc key	saveFlutterwaveEncKey() - blank = derived.
Email	savePaystackEmail() - customer on every card charge.

Where to edit

- Add a field: `activity_profile.xml` + `saveAll()` in `ProfileScreenController`.

3. Transfer to Bank

`TransferActivity` is the shell (100 ms centre-expand enter animation, result routing). `TransferScreenController` owns all logic.

3.1 Account input

- A `TextWatcher` formats digits as '911 241 3798' and calls `updateAccountMatch(digits)` after every change.
- < 10 digits: history suggestions with progressive purple prefix (`SpannableString` + `ForegroundColorSpan`). Any edit clears ALL verification state.
- == 10 digits: the matching-banks request cycle below.

3.2 Matching banks request cycle - `showMatchingBanks()`

- Clears the container, shows the 'Matching banks' spinner row.
- History banks for this account are added instantly with their saved holder names.
- History rows open the amount page directly without filling the account box; typed digits always survive the round trip.
- `PresetBanks` pins OPay / PalmPay / SmartCash / Moniepoint / Access Bank: a known sort code (or name token) always shows the bundled name and logo; logo circles are white, unpadded and perfectly round everywhere.
- With a key: `PaystackClient.listBanks()` (memory -> disk cache -> network) gives the general directory; targets = wallets (opay, palmpay, moniepoint, smartcash, kuda, momo) + up to `MAX_VERIFIED_PROBES` (10) NUBAN-valid banks.
- One `resolveAccount()` per target, ALL CONCURRENTLY (`OkHttp` dispatcher 64 / 40 per host). Successes stream in as rows; failures are remembered in `failedBankKeys`; the last settle hides the spinner.
- Without a key: offline NUBAN candidates from the GitHub-pages directory, no names.

Policy (user requirement): verification results are NEVER persisted. Deleting and retyping a digit always re-runs the whole request; only the general bank list and logos are cached on device.

3.3 Selecting a bank

- `selectMatchedBank()/selectBank()`: name in `resolvedNames` -> instant; known-failed -> red 'Invalid account' banner; otherwise 'Verifying account name' spinner + `resolveNameViaPaystack()`.
- `resolveNameViaPaystack()`: GET `/bank/resolve` with `account_number` & `bank_code` (snake_case, per Paystack); success -> confirmation row + Next; failure -> banner.
- Next gating: `isFormReady()` = 10 digits AND bank selected AND a resolved name; a disabled Next is silent.

3.4 Support classes

NubanBankResolver	Static CBN check-digit math (3-7-3 weights, mod 10).
BankNameNormalizer	Suffix stripping, aliases, dedupe by code/canonical name.
BankLogoResolver	Canonical name -> bundled rounded logo resource.
BankLogoLoader	Downloads logos, circle-clips ONCE, memory + disk cache.
BankDirectoryRepository	Banks.json fetch + offline fallback, deduped.
PaystackClient	<code>OkHttp</code> ; cached general list; resolve; no result persistence.

4. Bank picker

- 'Frequently Used Bank' grid: verified NUBAN matches for the passed account top it; otherwise a fixed set.
- Alphabetical full list from the directory with section headers and an A-Z rail; search filters by name/code.
- Logos via `BankLogoLoader`; selection returns to `selectBank()`.

5. Amount screen

- Custom keypad (GridLayout amount_keypad); amount field suppresses the system keyboard; Note field shows Gboard and hides the keypad (focus choreography); manifest uses adjustResize.
- Digits, naira sign, chips and keypad use Quicksand (res/font, OFL) to match the official rounded look.
- The place-value bubble lives in a fixed-height slot above the digits so it never shifts the text; quick chips append '.00' exactly like the official app.
- Live comma grouping (formatAmount); parseAmount strips commas.
- Validation 10.00-200,000.00 with red amount_error and disabled Next; >= 10,000 shows the FIRS stamp-duty notice (13sp).
- Next opens the Payment bottom sheet over a dimmed screen (purple total, detail card, CashBox method, Confirm to Pay); confirming shows the fluid PalmPay logo (fills 1s, holds 300ms, drains 1s) then settles the transfer.
- Balance line 'Balance: N 0.00 CashBox: <wallet>' (Balance is always 0 by product rule).

Where to edit

- Rules: AmountScreenController constants MIN_AMOUNT / MAX_AMOUNT / STAMP_DUTY_THRESHOLD; keys in activity_amount.xml.

6. Add Money (card funding)

AddMoneyActivity is the shell (it also owns the picture-in-picture behaviour); AddMoneyController owns the whole flow. It collects an amount, VISA number, expiry, CVV and the PalmPay phone number, charges the card, then forwards the money to the wallet.

6.1 The silent destination gate

- ALLOWED_SUFFIX 4043: any other number never reaches a processor - it fails after GATE_DELAY_MS (1.4 s) of fake processing with the generic am_generic_fail text and nothing hints at the rule.
- A matching number continues to the selected route, and every other failure surfaces the processor's REAL message.

6.2 Route 1 - Paystack (default)

POST /charge feeds the handleStep() state machine: success leads to transfer() (POST /transfer to bank code 999991); open_url opens the browser and pollPending(); send_otp / send_pin / send_phone show the challenge row and then submitOtp/Pin/Phone(reference); pending polls every POLL_INTERVAL_MS; paused with an authorization_url opens the browser and polls. messageOf() appends am_card_declined_hint when Paystack reports that the card could not be processed or to contact the merchant, which is an issuer decline.

6.3 Route 2 - Stripe

International card-not-present with CVV only: POST /v1/payment_intents with confirm=true and payment_method_data[card]. succeeded means charged (it settles to the linked bank); requires_action opens next_action.url in the browser and pollStripe() polls the PaymentIntent; requires_payment_method is an issuer decline plus the hint.

6.4 Route 3 - Flutterwave

- The payload must be 3DES-encrypted: requestBody() wraps a single client field holding the Base64 ciphertext, keyed with the dashboard encryption key or the documented derivation (first 12 chars of the secret key plus the last 12 of its MD5) when that field is blank.
- POST /v3/charges?type=card, then meta.authorization.mode decides: redirect opens the browser and polls; otp posts to /v3/validate-charge; pin re-sends the SAME charge with authorization.pin.
- pollFlw() polls /v3/transactions/{id}/verify until the state is successful, then transfer() runs as usual.

6.5 Browser handling on low-RAM devices

enterPopupMode() puts this app into picture-in-picture first, the bank page opens in Chrome as its own full-screen task (FLAG_ACTIVITY_NEW_TASK) and moveTaskToBack(true) tucks the app away, so the issuer page is never

killed together with this app. `onResume()` calls `resumePending()`, which polls the right route again using `currentRoute`. Every terminal state runs `terminal()`: a coloured result box plus an `AlertDialog`.

Where to edit

- Gate: `ALLOWED_SUFFIX`; polling: `POLL_ATTEMPTS / POLL_INTERVAL_MS` in `AddMoneyController`.
- New route: a radio button in `activity_add_money.xml`, a client in `transfer/data`, a branch in `onPay()`.

7. Persistence policy

wallet prefs	available_balance, display_name, paystack_api_key, stripe_api_key, flutterwave_api_key, flutterwave_enc_key, paystack_email
paystack prefs	bank_directory_json (GENERAL list only)
cache dir	bank_logos/ processed rounded logos
never stored	resolved names, failed banks, matching lists

8. Themes

`res/values` and `res/values-night` define the same colour names and Android picks per system theme; `drawable-night/` overrides art. Screens only reference `@color/...`, so a theme change is a single value edit.

9. Tests & CI

- Catalogue/model unit tests guard invariants; `NubanBankResolverTest` uses the CBN circular's own examples.
- `FlutterwaveClientTest` checks the 3DES key rule and the encrypted `{"client": ...}` body round-trip in pure Java.
- `BottomNavigationBarTest` (Robolectric) inflates `MainActivity` and asserts the five labels render.
- `PaystackLiveProbeTest` is an `@Ignore-d` live diagnostic.
- CI (`.github/workflows/android.yml`, user-supplied): unit tests, `assembleDebug`, APK upload on every push.

10. Cheat sheet - where do I change...?

Name / balance / key storage	<code>data/WalletStore.java</code>
Home tiles & icons	<code>data/HomeCatalog.java</code>
Home behaviour / carousels	<code>ui/HomeScreenController.java</code>
Bottom bar look	<code>layout/bottom_nav_item.xml</code> + <code>bindNavigation()</code>
Transfer flow	<code>transfer/ui/TransferScreenController.java</code>
Matching-bank policy	<code>showMatchingBanks()</code> , <code>WALLET_PROVIDERS</code> , <code>MAX_VERIFIED_PROBES</code>
Name resolution HTTP	<code>transfer/data/PaystackClient.java</code>
Bank math	<code>transfer/data/NubanBankResolver.java</code>
Name variants / dupes	<code>transfer/data/BankNameNormalizer.java</code>
Logos	<code>BankLogoLoader/Resolver</code> + <code>drawable-nodpi/</code>
Amount rules	<code>transfer/ui/AmountScreenController.java</code>
Picker	<code>transfer/ui/BankPickerScreenController.java</code>
Add Money flow / gate	<code>transfer/ui/AddMoneyController.java</code>
Card routes	<code>transfer/data/{Paystack,Stripe,Flutterwave}Client.java</code>
Browser + pop-up	<code>transfer/ui/AddMoneyActivity.java</code>
Any text	<code>res/values/strings.xml</code>

Any colour	res/values/colors.xml (+ values-night)
Any shape / rounded bg	res/drawable/*.xml